APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

Seventh Semester B.Tech Degree Regular and Supplementary Examination, December 2023 (2019 Scheme)

Course Code: AIT401 | FOUNDATIONS OF DEEP LEARNING

MODEL ANSWER SCRIPT (Full-Marks Answers) | Max. Marks: 100 | Duration: 3 Hours

*Note: in the examination Part B requires one full question from each module (Q11/12, 13/14, 15/16, 17/18, 19/20). As requested, answers to **all** questions are given here, with depth matched to the marks allotted (Part A: 3 marks each; Part B: 14 marks per question).*

PART A — Short Answers (3 marks each)

Q1. Equations of Sigmoid, Tanh, ReLU and their derivatives [3 marks]

Function	Equation	Derivative	Range / remark
Sigmoid	$\sigma(x) = 1 / (1 + e^{-x})$	$\sigma'(x) = \sigma(x)\cdot(1 - \sigma(x))$	(0, 1); max slope 0.25; saturates → vanishing gradient
Tanh	$\tanh(x) = (e^{x} - e^{-x}) / (e^{x} + e^{-x})$	$\tanh'(x) = 1 - \tanh^{2}(x)$	(−1, 1); zero-centred; max slope 1
ReLU	$f(x) = \max(0, x)$	$f'(x) = 1$ for $x > 0$; 0 for $x < 0$	[0, ∞); cheap, no saturation for $x > 0$ (derivative at 0 taken as 0 or 1)

Q2. Network with two single-neuron ReLU hidden layers and an output neuron to approximate $f(x) = x^2 - 4x + 4$ on [0, 5] [3 marks]

Note that $f(x) = (x - 2)^2$: it decreases on [0, 2] and increases on [2, 5]. Architecture: **x → h_1 → h_2 → y** with

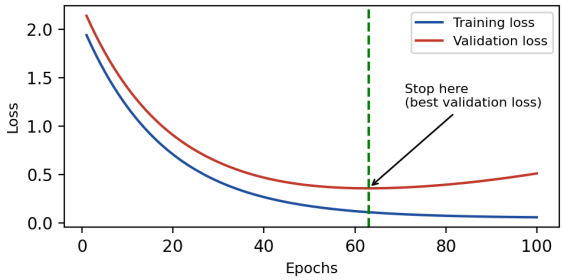
$$h_1 = ReLU(w_1 x + b_1), \quad h_2 = ReLU(w_2 h_1 + b_2), \quad y = w_3 h_2 + b_3$$

Chosen weights: $w_1 = 1$, $b_1 = -2$ (kink at x = 2, the minimum of f); $w_2 = 1$, $b_2 = 0$ (passes h_1 on); $w_3 = 3$, $b_3 = 0$. Hence $y = 3\cdot\max(0, x - 2)$: it is exact at x = 2 (f = 0) and x = 5 (f = 9) and follows the rising branch of the parabola with a straight-line piece.

Limitation (justification): a chain of single neurons is a composition of monotone functions, so the output is monotone in x, whereas f is not. One neuron per layer therefore cannot represent both branches; it can only fit one branch (here the right one) well. For an accurate fit one needs more neurons per layer: e.g. several parallel ReLU units with kinks at x = 1, 2, 3, 4 whose outputs are combined linearly by the output neuron, since ReLU networks give piecewise-linear approximations whose accuracy improves with the number of linear pieces (kinks).

Q3. Early stopping in neural network training [3 marks]

Early stopping is a regularization technique in which training is halted when the error on a held-out **validation set** stops improving, even though the training error keeps falling. After the point of minimum validation loss the network begins to memorize noise (overfitting). Procedure: evaluate validation loss after every epoch → keep a copy of the best weights → stop if no improvement for a fixed number of epochs (*patience*) → restore the best weights. It is simple, costs no extra parameters and limits the effective model capacity (similar in effect to L2 regularization).



Q4. Why is weight initialization crucial? [3 marks]

- **Breaks symmetry:** if all weights are equal (e.g. zero) every neuron in a layer gets the same gradient and learns the same feature.

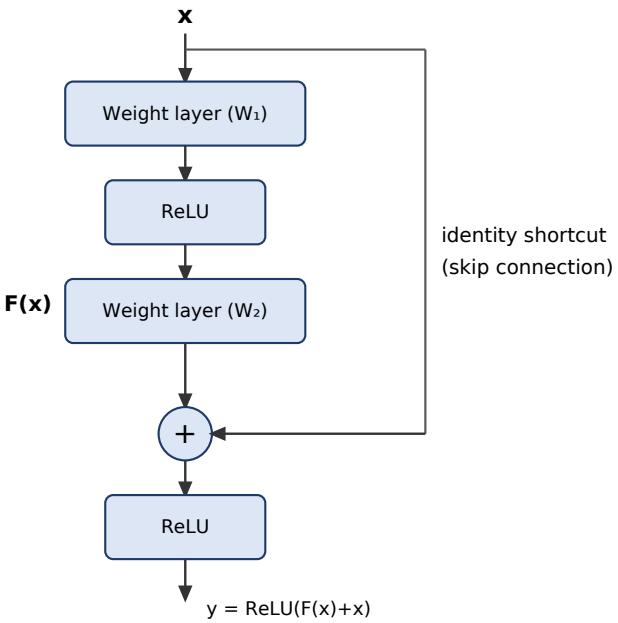
- **Controls signal flow:** too small weights → activations and gradients shrink layer by layer (*vanishing*); too large → they blow up (*exploding*) or saturate sigmoid/tanh units.
- **Speed and quality of convergence:** schemes such as Xavier (Var = $2/(n_{in}+n_{out})$) and He/Kaiming (Var = $2/n_{in}$) keep activation variance stable across layers, giving faster, more reliable training and better minima.

Q5. Residual block in deep neural networks [3 marks]

A residual block (ResNet) adds a **skip (identity) connection** around a few layers so that the layers learn a residual function F(x) = H(x) − x instead of the full mapping H(x):

$$y = F(x, \{W_i\}) + x$$

If dimensions differ, the shortcut uses a 1×1 projection $W_s x$. Since $\partial y/\partial x = \partial F/\partial x + 1$, gradients flow directly through the shortcut, which removes the vanishing-gradient and *degradation* problems and allows networks with 50–1000 layers to be trained; if identity is optimal the block just drives F → 0.



Q6. Effect of increasing stride in a convolutional layer; maximum stride [3 marks]

Output size = ⌊(N − K + 2P)/S⌋ + 1. Increasing the stride S moves the filter in larger jumps, so: (i) the output feature map becomes smaller (down-sampling, like pooling); (ii) computation and memory fall; (iii) the overlap between receptive fields reduces and the receptive field of later layers grows faster; (iv) some spatial information is lost, and too large a stride may skip pixels.

Maximum stride: to cover every input pixel the stride should not exceed the kernel size, i.e. $S_{max} = K$ (for a K×K filter). If S > K, pixels between successive windows are never seen by the filter. (Absolute upper limit: S = N gives a single 1×1 output.)

Q7. Function of the input gate in LSTM [3 marks]

The input gate decides **how much of the new candidate information is written into the cell state**. It is a sigmoid layer producing values between 0 (write nothing) and 1 (write fully):

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad \tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C), \quad C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Thus the input gate filters the candidate values $\tilde{C}_t$, protecting the memory cell from irrelevant inputs.

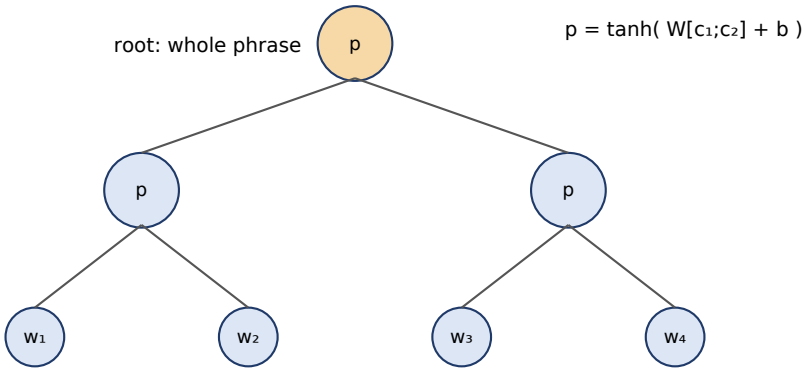
Q8. Recursive neural network [3 marks]

A recursive neural network applies the **same set of weights recursively over a tree (hierarchical) structure** instead of a chain. Child representations are merged bottom-up into a parent vector, up to the root which represents the whole input:

$$p = \tanh(W[c_1 ; c_2] + b)$$

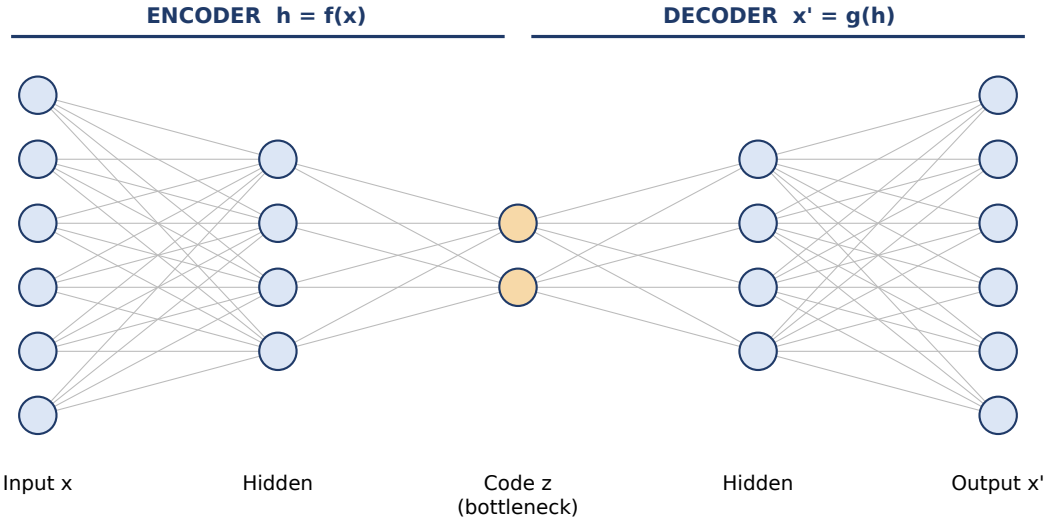
Used for parsing, sentence/phrase sentiment (parse trees) and scene parsing. The depth of a tree for n inputs is O(log n) rather than n, reducing long-term-dependency problems. A recurrent network is the special case of a

recursive network with a chain-shaped tree.



Q9. General structure of an autoencoder [3 marks]

An autoencoder is an unsupervised network trained to reproduce its input. It has an **encoder** h = f(x) that compresses the input into a low-dimensional **code (bottleneck)**, and a **decoder** x′ = g(h) that reconstructs the input from the code. It is trained to minimise the reconstruction loss L(x, x′) = ‖x − x′‖² so the code captures the most important features. Uses: dimensionality reduction, denoising, feature learning, anomaly detection.



Q10. Two applications of GAN [3 marks]

- **Image super-resolution / image-to-image translation:** SRGAN produces high-resolution, photo-realistic images from low-resolution ones; pix2pix/CycleGAN convert sketches to photos, day to night, horse to zebra.
- **Synthetic data generation / augmentation:** GANs create realistic faces, medical images or text-to-image samples to enlarge small datasets and train other models.

PART B — Module I

Q11.

(a) Advantages of a deep feedforward network over shallow networks [4 marks]

- **Hierarchical feature learning:** early layers learn simple features (edges), later layers combine them into complex ones (parts, objects) — raw data to abstract representation.
- **Parameter efficiency:** some functions need exponentially many units in a shallow (one hidden layer) network but only polynomially many in a deep one, because depth gives compositional re-use of features.
- **Greater expressive power and better generalization:** deep models represent highly non-linear functions with fewer parameters, which typically generalises better on vision, speech and language tasks.
- **End-to-end learning:** reduces manual feature engineering; features and classifier are learned jointly.

(b) Overfitting and underfitting [10 marks]

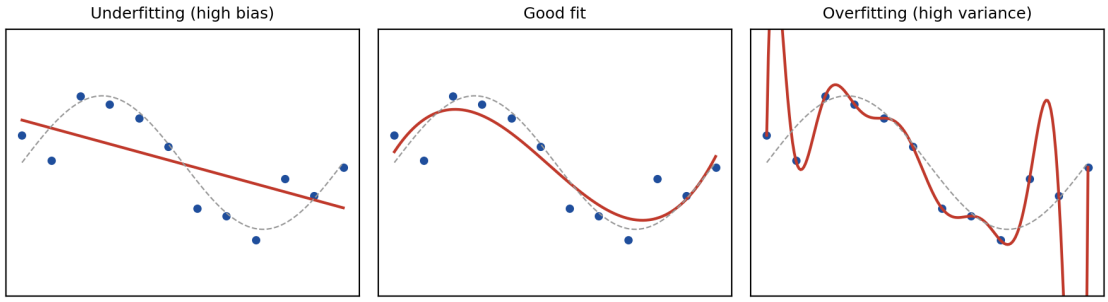
Generalization is a model's ability to perform well on unseen data. Training aims to make the training error small and the gap between training and test error small. Failure of either goal gives underfitting or overfitting.

Underfitting (high bias)

The model is too simple (or trained too little) to capture the underlying pattern, so error is high on **both** training and test data. *Causes:* too few layers/neurons, excessive regularization, too few epochs, poor features. *Remedies:* larger/deeper network, train longer, reduce regularization, better features, better optimizer.

Overfitting (high variance)

The model fits the training data, including its noise, extremely well (very low training error) but performs poorly on test data (large gap). *Causes:* excessive capacity, small or noisy dataset, training too long. *Remedies:* more data/data augmentation, L1/L2 regularization, dropout, early stopping, batch normalization, reduce model size, ensembling, cross-validation.



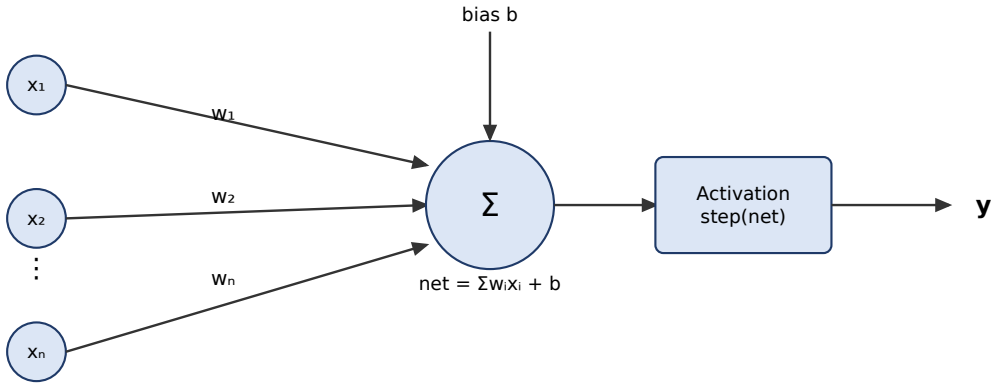
Aspect	Underfitting	Good fit	Overfitting
Training error	High	Low	Very low
Validation/test error	High	Low (close to training)	High
Bias / variance	High bias, low variance	Balanced	Low bias, high variance
Model complexity	Too low	Appropriate	Too high

Bias–variance trade-off: Expected error = Bias² + Variance + Irreducible noise. Increasing capacity reduces bias but increases variance; the aim is the capacity at which validation error is minimum. Detect both by plotting training and validation loss against epochs: both high → underfitting; training falling while validation rises → overfitting.

Q12.

(a) Structure and functionality of a perceptron [7 marks]

The perceptron (Rosenblatt, 1958) is the simplest artificial neuron and a binary linear classifier. **Structure:** (1) input vector x = (x_1 … x_n); (2) a weight w_i on each input, representing its importance; (3) a bias b (threshold shift); (4) a summing unit computing net = $\Sigma w_i x_i + b$; (5) a step activation function; (6) the output y.



$$y = 1 \text{ if } w \cdot x + b > 0; \quad y = 0 \text{ otherwise}$$

Functionality: the equation w·x + b = 0 is a hyperplane that divides the input space into two classes, so the perceptron solves linearly separable problems (AND, OR) but not XOR. **Learning rule** (for each training example, learning rate η, target t, prediction y):

$$w_i \leftarrow w_i + \eta\,(t - y)\,x_i, \quad b \leftarrow b + \eta\,(t - y)$$

Weights change only for misclassified samples; the perceptron convergence theorem guarantees convergence in finite steps when data are linearly separable. Stacking many perceptron-like units with non-linear activations gives the multilayer perceptron, which overcomes the XOR limitation.

(b) Role of the loss function; significance of choosing different loss functions [7 marks]

Role: The loss (cost) function L(y, ŷ) measures how far the network's prediction ŷ is from the target y. (i) It gives a single scalar objective that training minimises; (ii) its gradient ∂L/∂ŷ is the starting point of backpropagation, so it determines how weights are updated ($\theta \leftarrow \theta - \eta\nabla_\theta L$); (iii) it defines what 'good' means for the task and is used to monitor training and validation.

Loss	Formula	Used for / behaviour
Mean Squared Error	$(1/n)\Sigma(y - \hat{y})^2$	Regression; smooth, penalises large errors heavily (sensitive to outliers)
Mean Absolute Error	$(1/n)\Sigma\lvert y - \hat{y}\rvert$	Regression; robust to outliers, constant gradient
Huber	Quadratic for small error, linear for large	Regression; combines MSE and MAE advantages
Binary cross-entropy	$-[y \log \hat{y} + (1 - y)\log(1 - \hat{y})]$	Binary classification with sigmoid output
Categorical cross-entropy	$-\Sigma\, y_k \log \hat{y}_k$	Multi-class classification with softmax output
Hinge	$\max(0, 1 - y\cdot\hat{y})$	Maximum-margin classifiers (SVM-style)
KL divergence	$\Sigma\, p \log(p/q)$	Comparing distributions (VAE, distillation)

Significance of the choice: the loss must match the task and output activation (cross-entropy for probabilities, MSE for real values). It affects gradient behaviour (cross-entropy with softmax/sigmoid avoids the slow learning that MSE shows when units saturate), robustness to outliers, convergence speed, the probabilistic interpretation (MSE ⇔ Gaussian likelihood, cross-entropy ⇔ Bernoulli/categorical likelihood), class-imbalance handling (weighted losses) and the final accuracy. A regularization term ($\lambda\|w\|^2$) is often added to the loss to reduce overfitting.

PART B — Module II

Q13.

(a) Kaiming and Xavier weight initialization and their significance [6 marks]

Idea: choose initial weight variance so that the variance of activations (forward) and gradients (backward) stays about the same across layers, avoiding vanishing/exploding signals. For a layer $y = \Sigma w_i x_i$ with n_{in} inputs, $Var(y) = n_{in}\cdot Var(w)\cdot Var(x)$, so Var(w) must be $\propto 1/n_{in}$.

	Xavier (Glorot) initialization	Kaiming (He) initialization
Suited for	Sigmoid, tanh (symmetric, roughly linear near 0)	ReLU / Leaky ReLU (non-symmetric)
Variance	$Var(W) = 2 / (n_{in} + n_{out})$	$Var(W) = 2 / n_{in}$ (or $2/n_{out}$)
Normal	$W \sim N(0, 2/(n_{in}+n_{out}))$	$W \sim N(0, 2/n_{in})$
Uniform	$W \sim U(-\sqrt{(6/(n_{in}+n_{out}))}, +\sqrt{(6/(n_{in}+n_{out}))})$	$W \sim U(-\sqrt{(6/n_{in})}, +\sqrt{(6/n_{in})})$
Reasoning	Compromise between keeping forward variance ($1/n_{in}$) and backward gradient variance ($1/n_{out}$)	ReLU zeroes half of the activations, halving the variance, so the factor 2 compensates

Significance: they prevent vanishing/exploding gradients, break symmetry, allow deeper networks to train, and give faster and more stable convergence. Biases are generally initialised to zero.

(b) Regularization and various regularization techniques [8 marks]

Regularization is any modification to a learning algorithm intended to reduce its generalization (test) error but not its training error, i.e. to control overfitting by constraining model complexity. Main techniques:

- **L2 (weight decay / ridge):** $J = L + (\lambda/2)\|w\|^2$; shrinks weights, update $w \leftarrow w(1 - \eta\lambda) - \eta\partial L/\partial w$; gives small, smooth weights.
- **L1 (lasso):** $J = L + \lambda\Sigma|w|$; drives many weights exactly to zero → sparse model and feature selection.

- **Dropout:** during training each neuron is dropped with probability p, preventing co-adaptation; equivalent to training an ensemble of sub-networks; at test time all neurons are used with scaled weights.
- **Early stopping:** stop when validation loss starts rising.
- **Data augmentation:** flips, rotations, crops, noise enlarge the effective training set.
- **Batch normalization:** normalises layer inputs; its mini-batch noise has a mild regularizing effect.
- **Noise injection / label smoothing:** adding noise to inputs, weights or labels makes the model robust.
- **Parameter sharing and max-norm constraints, ensembling (bagging)** also reduce variance.

Q14.

(a) Gradients of a multiplicative neuron $o = x_1 \cdot x_2$ given $\partial L/\partial o = 5$, $x_1 = 2$, $x_2 = 3$ [7 marks]

By the chain rule, $\partial L/\partial x_i = (\partial L/\partial o)(\partial o/\partial x_i)$. For $o = x_1 x_2$: $\partial o/\partial x_1 = x_2$ and $\partial o/\partial x_2 = x_1$.

$$\partial L/\partial x_1 = (\partial L/\partial o) \cdot x_2 = 5 \times 3 = \mathbf{15}$$

$$\partial L/\partial x_2 = (\partial L/\partial o) \cdot x_1 = 5 \times 2 = \mathbf{10}$$

A multiplication gate acts as a *gradient switcher*: the gradient of each input is the upstream gradient multiplied by the other input.

(b) Gradient Descent vs Stochastic Gradient Descent vs Mini-batch Gradient Descent [7 marks]

All three update parameters as $\theta \leftarrow \theta - \eta \nabla_\theta J(\theta)$; they differ in how many samples are used to estimate the gradient.

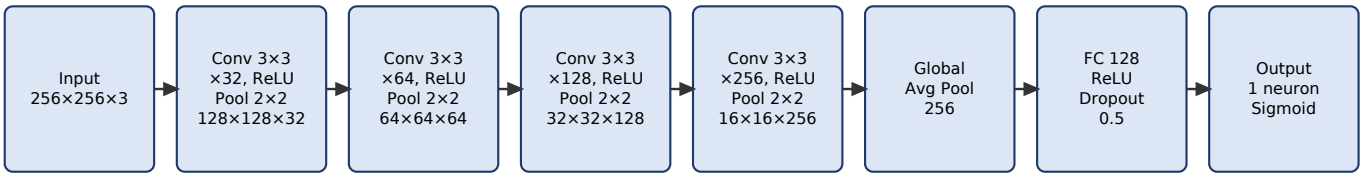
Aspect	Batch GD	Stochastic GD	Mini-batch GD
Samples per update	Entire dataset (N)	1 sample	Batch of B samples (e.g. 32–256)
Update rule	$\theta - \eta \cdot (1/N) \Sigma \nabla L_i$	$\theta - \eta \nabla L_i$ (random i)	$\theta - \eta \cdot (1/B) \Sigma_{batch} \nabla L_i$
Gradient estimate	Exact, no noise	Very noisy	Moderately noisy
Updates / epoch	1	N	N/B
Speed / memory	Slow, needs all data in memory	Fast per update, little memory; cannot vectorise	Fast, uses GPU vectorisation, moderate memory
Convergence	Smooth, stable; stuck in local minima/saddle	Oscillates, can escape shallow minima; needs decaying η	Stable and efficient; most widely used

PART B — Module III

Q15.

(a) CNN for gender classification on 256 × 256 face images [7 marks]

Gender classification is a binary problem, so the network ends with one sigmoid neuron trained with binary cross-entropy. Small 3×3 filters are used (stacked 3×3 filters give a large receptive field with fewer parameters), 'same' padding, ReLU activation, 2×2 max-pooling after each block, and the number of filters doubled as spatial size halves. Input: 256×256×3 (RGB).



Layer	Filter / size	Activation	Output (H×W×C)	Parameters
Input	–	–	256×256×3	0
Conv1 + MaxPool 2×2	3×3, 32 filters, stride 1, pad same	ReLU	128×128×32	3·3·3·32+32 = 896
Conv2 + MaxPool 2×2	3×3, 64 filters	ReLU	64×64×64	18,496
Conv3 + MaxPool 2×2	3×3, 128 filters	ReLU	32×32×128	73,856

Layer	Filter / size	Activation	Output (H×W×C)	Parameters
Conv4 + MaxPool 2×2	3×3, 256 filters	ReLU	16×16×256	295,168
Global Average Pooling	–	–	256	0
Fully connected + Dropout(0.5)	128 neurons	ReLU	128	32,896
Output	1 neuron	Sigmoid	1 (P(male))	129

Total $\approx 4.2 \times 10^5$ parameters. Batch normalization may be added after each convolution, the loss is binary cross-entropy, the optimizer Adam (lr = 10^{-3}), and data augmentation (flip, small rotation, brightness) reduces overfitting. Filter widths 32→64→128→256 learn edges → facial parts (eyes, nose, jaw) → high-level facial structure.

(b) Motivation for convolutional layers; advantages over fully connected layers [7 marks]

Images are high-dimensional with strong **local spatial correlation** and the same patterns (edges, textures) occur anywhere. Convolutional layers exploit this structure:

- **Sparse (local) connectivity:** each output unit sees only a small receptive field (e.g. 3×3), not the whole image.
- **Parameter sharing:** the same filter is slid over all positions, so a feature learned in one place is detected everywhere.
- **Translation equivariance:** shifting the object shifts the feature map; with pooling gives approximate translation invariance.
- **Far fewer parameters:** for a 256×256×3 image (196,608 inputs), a fully connected layer of 1000 neurons needs ≈ 196.6 million weights, whereas a conv layer with 64 filters of 3×3×3 needs only 1,728 weights. This reduces memory, computation and overfitting.
- **Hierarchical features:** stacked convolutions build edges → textures → parts → objects; FC layers ignore spatial structure and treat shuffled pixels the same.
- **Works for any input size** (fully convolutional), while FC layers need a fixed size.

Q16.

(a) Output size for 28×28 input, 3×3 kernel, padding 0, stride 2; padding for same size with stride 1 [4 marks]

$$\textit{Output size} = \lfloor (N - K + 2P)/S \rfloor + 1$$

(i) N = 28, K = 3, P = 0, S = 2: $\lfloor (28 - 3 + 0)/2 \rfloor + 1 = \lfloor 12.5 \rfloor + 1 = 12 + 1 =$ **13**. Output feature map = **13 × 13**.

(ii) For the same size with S = 1: $(28 - 3 + 2P)/1 + 1 = 28 \Rightarrow 25 + 2P = 27 \Rightarrow$ **P = 1** (in general P = (K − 1)/2 = 1).

(b) Differences between AlexNet, ZFNet, VGG-19 and ResNet-50 [10 marks]

The four networks are ImageNet (ILSVRC) milestones; they are compared below.

Feature	AlexNet (2012)	ZFNet (2013)	VGG-19 (2014)	ResNet-50 (2015)
Depth	8 layers (5 conv + 3 FC)	8 layers (5 conv + 3 FC)	19 layers (16 conv + 3 FC)	50 layers (conv + 1 FC)
Input size	227×227×3 (224)	224×224×3	224×224×3	224×224×3
Parameters	≈ 60 M	≈ 60 M	≈ 144 M	≈ 25.6 M
Filters	11×11 (s=4), 5×5, 3×3	7×7 (s=2) first layer, then 5×5, 3×3	Only 3×3 conv (stride 1) and 2×2 max-pool	1×1, 3×3, 1×1 bottleneck blocks, one 7×7 stem
Key ideas	ReLU, dropout, data augmentation, overlapping pooling, LRN, training on 2 GPUs	Tuned AlexNet: smaller first-layer filter and stride, more filters in later layers; deconvnet visualization	Very deep, uniform and simple stack of 3×3 filters; two 3×3 ≈ one 5×5 receptive field with fewer parameters	Residual (skip) connections, batch normalization, global average pooling; no large FC layers
ILSVRC top-5 error (approx.)	≈ 15.3 % (winner 2012)	≈ 11.7 % (winner 2013)	≈ 7.3 % (runner-up 2014)	≈ 5.3 % single model (ResNet ensemble 3.57 %, winner 2015)

Feature	AlexNet (2012)	ZFNet (2013)	VGG-19 (2014)	ResNet-50 (2015)
Strength / weakness	First deep CNN success; large FC layers → many parameters	Better than AlexNet via visualisation-guided design	Excellent features, but huge memory and slow	Trains very deep nets, accurate and efficient; fewer parameters than VGG

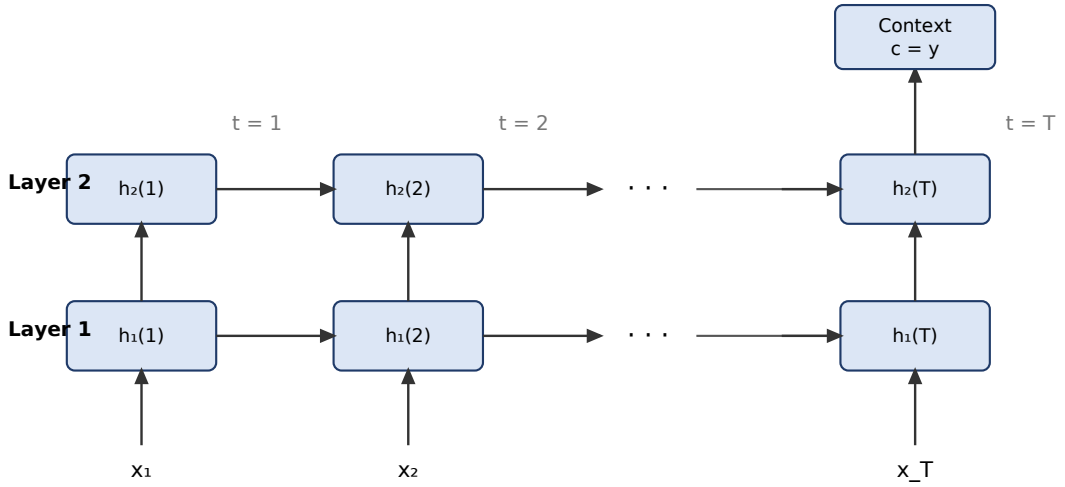
Trend: accuracy rose with depth, smaller filters and better optimisation — AlexNet → ZFNet (better filter design) → VGG (depth with 3×3 filters) → ResNet (skip connections solving the degradation problem, with fewer parameters than VGG).

PART B — Module IV

Q17.

(a) Multi-layered RNN taking a sequence of length T and producing a single output (context) [7 marks]

A **stacked (deep) RNN** with L layers is unrolled over T time steps. The input x_t feeds layer 1; the hidden state of each layer is passed upward to the next layer at the same time step and forward in time within the layer (weights are shared across time). Only the **final hidden state of the top layer** is used as the context/output (many-to-one).



$$h_t^{(1)} = \tanh(W_x^{(1)} x_t + W_h^{(1)} h_{t-1}^{(1)} + b^{(1)})$$

$$h_t^{(l)} = \tanh(W_x^{(l)} h_t^{(l-1)} + W_h^{(l)} h_{t-1}^{(l)} + b^{(l)}),\ l = 2 \dots L,\ h_0 = 0$$

$$\text{Context } c = h_T^{(L)},\quad y = \text{softmax}(W_y c + b_y)$$

Training uses backpropagation through time (BPTT) with loss computed only at step T; gradient clipping handles exploding gradients, and LSTM/GRU cells can replace tanh units for long sequences. The context c summarises the entire sequence (e.g. for sentiment classification or as the encoder output in sequence-to-sequence models).

(b) How LSTM addresses long-term dependency issues in language models [7 marks]

In a vanilla RNN, gradients are repeatedly multiplied by W_h and $\tanh'(\cdot) < 1$ while backpropagating through time, so they **vanish** (or explode) and the network cannot connect words that are far apart (e.g. 'The cats, which … , are' – subject–verb agreement). LSTM introduces a **cell state C_t** – a memory 'conveyor belt' – regulated by three gates:

- **Forget gate:** $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$ – what to erase from memory.
- **Input gate:** $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$, candidate $\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$ – what new information to store.
- **Output gate:** $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$, $h_t = o_t \odot \tanh(C_t)$ – what to expose as hidden state.

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Because the cell state is updated **additively** (not by repeated matrix multiplication), $\partial C_t / \partial C_{t-1} = f_t$; when $f_t \approx 1$ the gradient flows unchanged over many steps (the 'constant error carousel'), so information such as the grammatical subject can be kept for dozens of words and used when predicting the verb. The gates learn when to remember, update or forget, which lets LSTM language models capture long-range context far better than

simple RNNs.

Q18.

(a) Example of RNNs in language modeling; how RNNs generate sequences [7 marks]

A language model estimates $P(w_1 \dots w_T) = \Pi\, P(w_t \mid w_1 \dots w_{t-1})$. An RNN does this by keeping the history in its hidden state. **Example:** a word-/character-level next-word predictor (keyboard suggestion, speech recognition, machine translation decoder). At each step:

$$h_t = tanh(W_{xh}x_t + W_{hh}h_{t-1} + b), \quad \hat{y}_t = softmax(W_{hy}h_t + c) = P(w_{t+1} \mid w_{\le t})$$

Example: for 'the cat sat on the', the model outputs a distribution over the vocabulary with high probability for 'mat'. **Training:** inputs are one-hot/embedding vectors, the target at step t is the next word, loss = Σ cross-entropy, trained by BPTT using *teacher forcing* (true previous word fed as input).

Sequence generation: (1) feed a start token <START> (or a seed text) and $h_0 = 0$; (2) compute the softmax distribution over the vocabulary; (3) **sample** (or take argmax / beam search; a temperature controls randomness) the next word w_{t+1}; (4) feed this generated word back as the next input; (5) repeat until <END> or a maximum length is reached. The RNN thus generates text, music or handwriting one element at a time, each element conditioned on all earlier ones through h_t.

(b) Important design patterns of RNNs [7 marks]

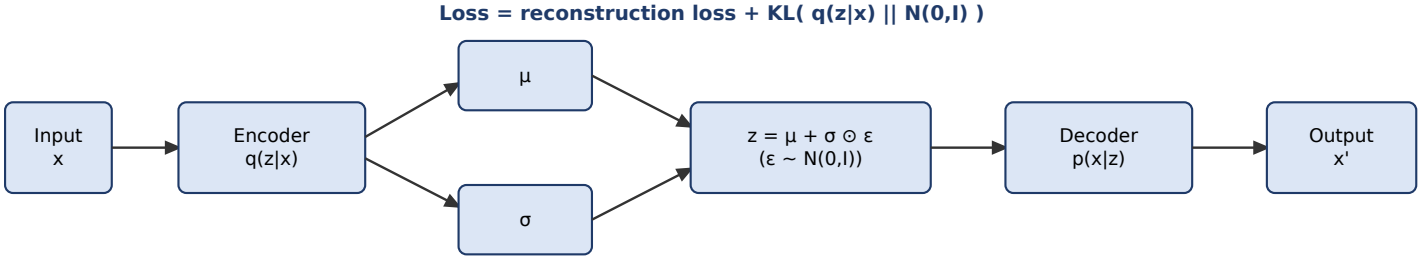
- **Pattern 1 – output at every time step, recurrence between hidden units:** $h_{t-1} \rightarrow h_t$ and an output o_t for each x_t (sequence labelling, language models; many-to-many, Turing-complete).
- **Pattern 2 – output at every step, recurrence only from output to hidden:** $o_{t-1} \rightarrow h_t$; less powerful but trainable in parallel using teacher forcing.
- **Pattern 3 – recurrence between hidden units, single output after reading the whole sequence:** many-to-one (sentiment classification, summarising a sequence to a fixed-size representation).
- **One-to-many** (image captioning: one image → word sequence), **many-to-many encoder–decoder / seq2seq** (machine translation, input and output lengths differ).
- **Bidirectional RNNs:** a forward and a backward RNN so the output at t uses both past and future context (speech/NER).
- **Deep (stacked) RNNs** and gated units (LSTM, GRU) for deeper representations and long-term memory; **recursive** and attention-based variants for tree data and long inputs.

PART B — Module V

Q19.

(a) Variational Autoencoders (VAE) [7 marks]

A VAE is a **generative** autoencoder that learns a probability distribution over a continuous latent space, so new data can be generated by sampling. Unlike a plain autoencoder whose encoder gives a single code, the VAE **encoder outputs the parameters (mean μ and standard deviation σ) of a Gaussian q(z|x)**; a latent vector z is sampled and the **decoder** p(x|z) reconstructs x.



Reparameterization trick: sampling is not differentiable, so write $z = \mu + \sigma \odot \varepsilon$ with $\varepsilon \sim N(0, I)$; randomness is moved to ε, letting gradients flow to μ and σ through backpropagation. **Loss (negative ELBO):**

$$L = E_{q(z|x)}[\, -log\ p(x|z)\ ] + KL(\ q(z|x) \parallel p(z)\ ), \quad p(z) = N(0, I)$$

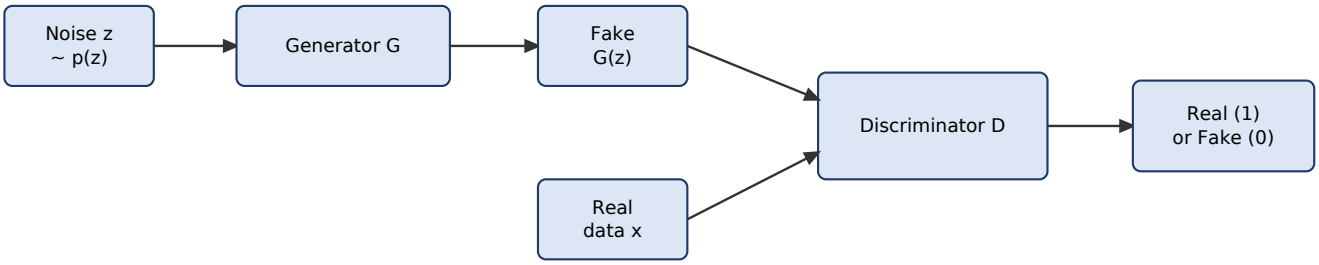
The first term is the reconstruction loss; the KL term regularises the latent space to be close to a standard normal, making it smooth and continuous. **Generation:** sample z ~ N(0, I) and pass through the decoder. **Applications:** image generation, interpolation between samples, denoising, anomaly detection, drug-molecule design.

(b) Applications of GAN [7 marks]

A GAN consists of a generator G (makes fake samples from noise z) and a discriminator D (distinguishes real from fake) trained in a minimax game:

$$\min_G \max_D V(D,G) = E_{x\sim p_{data}}[\log D(x)] + E_{z\sim p_z}[\log(1 - D(G(z)))]$$

G tries to fool D; D tries to tell real from fake



- **Realistic image generation:** faces, art, products (StyleGAN, BigGAN).
- **Image super-resolution (SRGAN)** and image enhancement/restoration.
- **Image-to-image translation:** pix2pix, CycleGAN – sketch → photo, day → night, season/style transfer.
- **Text-to-image synthesis** (e.g. StackGAN) and image inpainting (filling missing regions).
- **Data augmentation:** synthetic medical images (MRI, X-ray) and rare-class samples to train classifiers.
- **Face ageing, face editing, video generation, deepfakes** and 3-D object generation.
- **Drug and material discovery, music/speech synthesis, anomaly detection** (e.g. AnoGAN).

Q20.

(a) Boltzmann Machines: learning and probabilistic sample generation [7 marks]

A Boltzmann machine (BM) is a **stochastic recurrent network of symmetrically connected binary units** (visible units v and hidden units h) that models a probability distribution. Every configuration s = (v, h) has an **energy**, and probability follows the Boltzmann distribution:

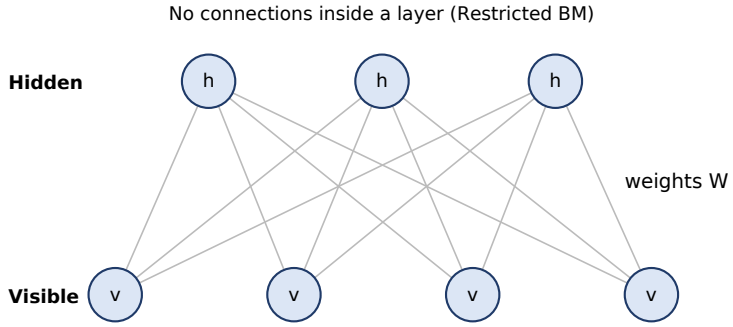
$$E(v,h) = -\Sigma_{i<j} w_{ij}s_i s_j - \Sigma_i b_i s_i, \quad P(v,h) = e^{-E(v,h)} / Z, \quad Z = \Sigma\, e^{-E}$$

Low-energy states are more probable. Each unit switches on stochastically: $P(s_i = 1) = \sigma(\Sigma_j w_{ij}s_j + b_i)$. Hidden units capture dependencies among visible units.

Learning: the weights are adjusted to maximise the log-likelihood of the training data. The gradient has a simple local form:

$$\Delta w_{ij} = \eta\,(\langle s_i s_j\rangle_{data} - \langle s_i s_j\rangle_{model})$$

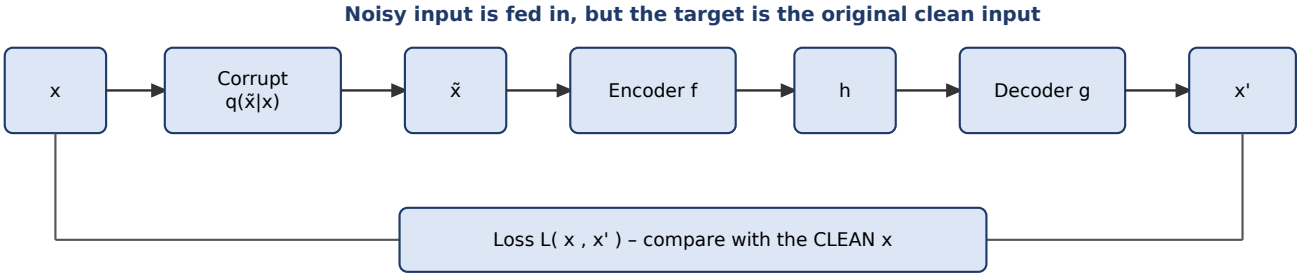
The *positive phase* clamps visible units to data and samples the hidden units; the *negative phase* lets the network run freely (Gibbs sampling) to estimate model statistics. Since computing Z and the model expectation is intractable, a sampling approximation is used; the **Restricted Boltzmann Machine (RBM)** removes visible–visible and hidden–hidden connections, giving a bipartite graph so that units in a layer are conditionally independent and trained efficiently by **Contrastive Divergence (CD-k)**: v → h → v′ → h′, $\Delta w = \eta(vh^T - v'h'^T)$.



Generating samples: start from a random state, repeatedly sample hidden units given visible units and visible given hidden (Gibbs sampling) – optionally with simulated annealing (gradually lowering temperature) – until the chain reaches equilibrium; the visible vector then is a sample from the learned distribution. Stacked RBMs form Deep Belief Networks.

(b) Denoising Autoencoders (DAE) [7 marks]

A denoising autoencoder is trained to **reconstruct the clean input from a corrupted version**. The input x is first corrupted into $\tilde{x} \sim q(\tilde{x}|x)$; the encoder maps it to $h = f(\tilde{x})$, the decoder gives $x' = g(h)$, and the loss compares the output with the **original clean x**, not with $\tilde{x}$:



$$L = \| x - g(f(\tilde{x})) \|^2 \quad \textit{(or cross-entropy for binary inputs)}$$

- **Corruption types:** additive Gaussian noise, masking noise (randomly set a fraction of inputs to zero), salt-and-pepper noise.
- **Why it works:** the identity mapping no longer minimises the loss, so even an over-complete network cannot simply copy its input. It must learn the structure of the data (the manifold on which clean data lie) and map corrupted points back onto it; it implicitly learns the score $\nabla_x \log p(x)$.
- **Advantages:** robust features, acts as a regularizer, avoids trivial solutions. Stacked DAEs can be used for pre-training deep networks.
- **Applications:** image/audio denoising, inpainting, feature learning for classification, anomaly detection.